

Mappeoppgave: Dagbok

IDATx1003 Programmering 1, høsten 2025

v3.0

Sammendrag

Mappeoppgave for emnet IDATx1003 Programmering 1 for studieprogrammet Bachelor i ingeniørfag, data (dataingeniør) ved NTNU Trondheim, Gjøvik og Ålesund.

Revisjonshistorikk

Versjon	Dato	Forfatter	Beskrivelse
1.0	2025-09-30	Arne Styve m.fl.*	Første utgave
1.1	2025-10-17	Arne Styve	Oppdatert kap 3.6.5 med riktige versjoner. Liten presisering i kap 3.3.2
2.0	2025-10-20	Arne Styve m.fl.*	Utvidet dokumentet med Del 2 (av 3) i kapittel 5 .
3.0	2025-11-04	Arne Styve m.fl.*	Utvidet dokumentet med Del 3 (av 3) i kapittel 6 .

(* Arne Styve, Cecilia Norevik Bratlie, Kiran Raja)

Innhold

1 Om dokumentet	4
2 Om mappevurdering i IDATx1003 Programmering 1	5
3 Prosjektoppgaven - Dagbok	6
3.1 Nivåer	6
3.2 Prosjektrapport	6
3.3 Kravspesifikasjon	6
3.3.1 Funksjonelle krav for Nivå 1	6
3.3.2 Funksjonelle krav for Nivå 2	7
3.3.3 Egne utvidelser	7
3.4 Avgrensninger	7
3.5 Bruk av KI verktøy	7
3.6 Øvrige krav til prosjektet (ikke-funksjonelle krav)	8
3.6.1 Kodekvalitet	8
3.6.2 Enhetstesting	8
3.6.3 Unntakshåndtering	8
3.6.4 Versjonskontroll	8
3.6.5 Java-biblioteker og Maven-plugins (valgfritt i Trodheim)	8
4 Mappe del 1 (av 3)	12
4.1 Kodeprosjekt	12
4.2 Rapport	13
5 Mappe Del 2 av 3	15
5.1 Nivå 1 - Minimal Viable Product (MVP)	15
5.1.1 Enhetstest av registerklassen for dagbokinnlegg	15
5.1.2 Oppdatere applikasjonsklassen	15
5.2 Nivå 2: Utvidet funksjonalitet – Eget register for forfattere m.m.	16
5.2.1 Enhetstest av de nye klassene	16
5.2.2 Oppdatere applikasjonsklassen	16
6 Mappe del 3 av 3	18
6.1 Hvilke læringsmål vi ønsker å teste i del 3	18
6.2 Krav til del 3	19
6.2.1 Krav til Programmet/koden	19
6.2.2 Git og GitHub	19
6.3 Krav til rapport	19

6.3.1	Noen kommentarer til rapportmalen	19
6.4	Bruk av AI-verktøy som GitHub CoPilot og ChatGPT	20
7	Viktige sjekkpunkter	21
7.1	Nyttige referanser	21
A	Vedlegg	23
A.1	Nyttige verktøy	23
A.1.1	UML-modellering	23
B	KI-deklarasjon	23

Kodelister

1	Eksempel på en pom.xml-fil med riktige versjoner	10
---	--	----

1 Om dokumentet

Dette dokumentet inneholder 3 deler:

Om mappevurdering - her er det en detaljert beskrivelse av hvordan mappevurdering gjennomføres i emnet, hvilke deler mappen består av, og hvordan mappen skal leveres

Kravspesifikasjon - Beskrivelse av prosjektet i sin helhet

Mappen Del 1 - Hva som skal gjøres i del 1 av 3 - NB! LES DENNE GRUNDIG FØR DU STARTER MED KODINGEN!!

2 Om mappevurdering i IDATx1003 Programmering 1

I dette emnet er det ingen skriftlig eller muntlig eksamen. Istedenfor benyttes mappe som vurderingsform. Mappen skal løses **individuellt**. Mappen består av et programmeringsprosjekt samt en rapport.

Mappen leveres ut rundt uke 40 og går parallelt med øvrige læringsaktiviteter ut semesteret. Detaljert prosjektbeskrivelse blir publisert i 3 deler:

- Del 1 - cirka uke 40: Fokuserer på design av entity-klasse(r) samt versjonskontroll og enhets-testing.
- Del 2 - cirka uke 43: Fokuserer på samling(er) og forretningslogikk.
- Del 3 - cirka uke 45: Fokuserer på ferdigstillelse av applikasjon med et tekstbasert brukergrensesnitt og komplette enhetstester. I tillegg skal dere utvide løsningen med egne forslag/ideer.

Hver ny del bygger på foregående del. Når en ny del publiseres, vil dere få muntlig tilbakemelding på arbeidet som er gjort så langt, med mulighet til å justere/endre/forbedre løsningen. Dere vil få i alt 3 slike tilbakemeldinger, der den siste tilbakemeldingen gis i uke 47.

Endelig fungerende løsning leveres i GitHub/GitLab og som ZIP-fil sammen med en rapport i Inspira innen fristen (se Blackboard for detaljer om innlevering).

Vekting mellom rapport og prosjekt mot endelig karakter:

- Rapport - 30%
- Prosjekt - 70%

Kilder:

- Hva [forskriften sier om mappe som vurderingsform](https://lovdata.no/dokument/SF/forskrift/2015-12-08-1449/KAPITTEL_5#%C2%A75-11):
https://lovdata.no/dokument/SF/forskrift/2015-12-08-1449/KAPITTEL_5#%C2%A75-11

3 Prosjektoppgaven - Dagbok

NB! Her følger en beskrivelse av den endelige løsningen som skal leveres til slutt i prosjektet. Senere i dokumentet er det en detaljert beskrivelse av hva dere skal utvikle i hver av de 3 delene. Ikke start på utviklingen/kodingen uten å lese hva som kreves i hver del!

Det skal utvikles en dagbok-type løsning der brukeren kan registrere sine daglige notater og refleksjoner. Dagboken kan gjerne være spisset mot et tema, som for eksempel *treningsdagbok*, *fiskedagbok* eller lignende. Dette velger du selv.

For å støtte oppstarten av prosjektet gis det eksempler på en *fiskedagbok*.

Applikasjonen skal være en enkeltstående applikasjon som kjører på **en** datamaskin (altså ingen nettverk). Derfor må navnet til forfatteren av dagbokinnlegget legges inn for hvert innlegg, og man kan legge inn ulike forfattere på de ulike innleggene.

3.1 Nivåer

Ønsket funksjonalitet i applikasjonen er inndelt i 2 nivå:

- **Nivå 1:** Et «**Minimum Viable Product**»-nivå (**MVP**). Består av helt grunnleggende funksjonalitet som det er forventet at alle skal kunne klare å implementere.
- **Nivå 2: Utvidet funksjonalitet.** Her vil det gis noen utfordringer (fortsatt innenfor pensum) som kanskje ikke alle vil klare å implementere.

For å oppnå full score (og høyeste karakter) forventes at Nivå 2 er implementert. Kravene til nivå 1 og 2 følger i kapittelet «Kravspesifikasjon» under.

3.2 Prosjektrapport

Det skal skrives en rapport for prosjektet. I rapporten skal dere forklare hvordan løsningen er bygget opp, hvilke valg som er tatt underveis, hvordan dere har anvendt designprinsipper og så videre.

Rapporten skal være på **ca. 2500–3000 ord**. I tillegg kommer figurer og eventuelt små kodesnutter for å vise hvordan utvalgte problemer er løst. Rapportens språk er enten norsk eller engelsk.

Det er lurt å starte på rapporten allerede fra starten. Dokumentmal er tilgjengelig på Blackboard.

3.3 Kravspesifikasjon

En kravspesifikasjon består gjerne av to hovedkategorier av krav:

- **Funksjonelle krav** - dette er krav til brukerfunksjonalitet, altså hva bruker av applikasjonen skal være i stand til å utføre med løsningen.
- **Ikke-funksjonelle krav** - dette er øvrige krav som for eksempel *brukervennlighet*, krav til *teknologier* som skal benyttes, krav til *responstid*, *robusthet* og så videre.

Følgende funksjonelle krav gjelder, fordelt på nivåer.

3.3.1 Funksjonelle krav for Nivå 1

Her forventes det at løsningen inneholder kun **en** samling (register) av dagbok-innlegg.

En bruker skal:

- kunne opprette et innlegg i dagboken. Innlegget skal inneholde informasjon som:
 - Navnet til bruker/forfatter
 - En tittel på innlegget, for eksempel Storfangst på fjorden i dag"
 - En lengre tekst (selve innlegget)
 - Dato og klokkeslett for innlegget
- kunne skrive ut alle innleggene på en strukturert og oversiktlig måte
- kunne søke etter innlegg på en bestemt dato
- kunne slette innlegg

3.3.2 Funksjonelle krav for Nivå 2

Her forventes at det lages et eget register over **forfattere**. Hver gang et nytt innlegg skrives, kan for eksempel brukeren få opp en liste over tidligere forfatter der bruker kan velge en av disse, eller legge til en ny forfatter.

En bruker skal:

- kunne velge blant tidligere forfattere når nytt innlegg opprettes
- kunne søke etter innlegg i et bestemt tidsrom (fra-dato, til-dato) og vise innleggene
- kunne søke etter innlegg fra en bestemt forfatter og vise innleggene
- kunne søke etter innlegg basert på et ord i innlegg og vise innleggene.
- kunne hente ut statistikk i form av antall innlegg pr forfatter. Kan for eksempel vises som en tabell der kolonne 1 er en liste av forfattere med antall innlegg pr forfatter i kolonne 2.

3.3.3 Egne utvidelser

I tillegg til de definerte funksjonelle kravene kan man utvide eller endre løsningen etter egne ideer og ønsker. Egne utvidelser vil telle positivt i vurderingen av besvarelsen.

3.4 Avgrensninger

Følgende avgrensninger av oppgaven gjelder:

- Applikasjonen skal være tekstbasert (ikke grafisk brukergrensesnitt).
- Applikasjonen skal kunne kjøres fra IDE og fra kommandolinje
- Programmeringsprosjektet **skal** være satt opp med **Maven** som byggsystem (Maven er valg fritt i Trondheim)

3.5 Bruk av KI verktøy

Det er tillatt å anvende KI verktøy som støtte for å løse mappen, så lenge:

- KI-verktøyet benyttes som sparringpartner/rådgiver og oppslagsverk, og ikke for å generere kode for direkte bruk
- eventuell kode som foreslås av KI-verktøyet gjennomgås kritisk og eventuelt bearbeides før dere tar koden i bruk i prosjektet
- dere kommenterer i koden der dere har fått hjelp av KI-verktøy
- bruken av KI-verktøy dokumenteres i rapporten med en egen KI-deklarasjon (hvilke verktøy ble benyttet til hva, og gjerne med eksempler på prompts- og svar)

Eksempler på KI-verktøy:

GitHub Copilot: Bruk chatbot'en, og ikke «code completion». Chatboten er inkludert i plugin for IntelliJ og VSCode. GitHub Copilot er mye bedre enn ChatGPT i forhold til programmeringskode. Studenter får gratis tilgang til GitHub CoPilot ved å søke om «educational license» hos GitHub (<https://education.github.com/pack>)

Microsoft CoPilot: Tilsvarende ChatGPT. Er det KI-verktøyet som formelt er godkjent av NTNU. Finnes ikke som plugin til IntelliJ.

Code Tutor by Kahn Academy: En KI-plugin til ChatGPT som tar rollen som tutor", ved å gi hint og veiledning istedenfor å generere løsningen. Se: <https://chat.openai.com/g/g-hRCqiqVLM-tutor-me>

3.6 Øvrige krav til prosjektet (ikke-funksjonelle krav)

Følgende krav og betingelser gjelder for hele mappen:

3.6.1 Kodekvalitet

Det forventes høy kodekvalitet i prosjektet. Bruk verktøy som **CheckStyle** (med Google sine regler) og **SonarLint**. Begge verktøy finnes som plugins til IntelliJ og VSCode med flere (se vår [emnewiki](#) for detaljer).

3.6.2 Enhetstesting

Det skal skrives enhetstester for den delen av koden som er forretnings-kritisk (forretnings-logikken), altså for den koden som er viktigst for å oppfylle sentrale krav. Feil her vil få store negative konsekvenser for programmet. Negativ testing er også viktig. Det er ikke fornuftig å lage enhetstester til klasser som utgjør brukergrensesnittet, med mindre det finnes metoder som kan testes uten at en bruker er involvert.

3.6.3 Unntakshåndtering

Uønskede hendelser og tilstander som forstyrrer normal flyt skal håndteres på en god måte. Håndteringen skal være rimelig og balansert (ikke for mye, ikke for lite) med det formål å gjøre koden mer robust. Vi gjør oppmerksom på at unntakshåndtering ikke er eksplisitt angitt i oppgavene under. Dere må selv avgjøre når og hvordan unntak skal implementeres. En viktig sammenheng der unntak bør brukes er validering av parametre til metoder.

3.6.4 Versjonskontroll

Prosjektet skal legges under versjonskontroll. Krav til versjonskontroll er ytterligere beskrevet i del 1.

3.6.5 Java-biblioteker og Maven-plugins (valgfritt i Trodheim)

Følgende versjoner av Java JDK, biblioteker og Maven plugins skal benyttes (versjoner som var tilgjengelige pr august 2025):

- Java: Siste tilgjengelige LTS (Long Time Support) versjon **skal** benyttes: versjon 21 (som var gjeldende ved semesterstart)
- Bibliotek (<dependencies> i pom.xml)
 - JUnit: *org.junit.jupiter:junit-jupiter-engine:5.13.4*
- MavenPlugins (<build><plugins> i pom.xml) - er valg fritt i Trondheim
 - *org.apache.maven.plugins:maven-compiler-plugin:3.14.0*
 - *org.apache.maven.plugins:maven-surefire-plugin:3.5.4*

For andre biblioteker dere måtte velge/trengte, husk å **alltid** spesifisere eksakt versjon, **aldri** bruke RELEASE eller LATEST!

Kodeliste 1: Eksempel på en pom.xml-fil med riktige versjoner

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0,http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- TODO: Replace with your own groupId and artifactID -->
  <groupId>edu.ntnu.iir.bidata</groupId>
  <artifactId>IDATA1003-asty-dagbok</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <!-- Per August 2025 the LTS version is 21 -->
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.13.4</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- The Java compiler -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.0</version>
      </plugin>

      <!-- The unit-test engine Surefire -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.3</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Del 1 (av 3)

Prosjektoppsett og grunnleggende klasser.

4 Mappe del 1 (av 3)

I første del av mappen er fokuset å:

- forstå oppgaven/prosjektet
- få opprettet prosjektet som et Maven-prosjekt
- sette prosjektet under versjonskontroll koblet til GitHub
- implementere entitetsklassen (klassen som representerer et dagbok-innlegg)
- implementere testklasse for å teste entitetsklassen.
- starte på rapporten

Detaljene følger under.

4.1 Kodeprosjekt

Dere får tilgang til et ferdig oppsatt prosjekt via **GitHub Classroom** for mappen (**blir annonsert på BlackBoard**). Prosjektet er satt opp som et **Maven**-prosjekt. For å få tilgang til dette prosjektet og få opprettet sitt eget repository for Mappen på GitHub, se BlackBoard og siden “Opprette Mappeprosjektet fra GitHub” under menyen “Mappevurdering” i BlackBoard.

Når prosjektet er åpnet og klonet fra Git, last inn og åpne prosjektet i et utviklingsmiljø (IntelliJ, VSCode eller lignende). Følg videre instruksjoner under (NB! Dere kan ikke bruke BlueJ på dette prosjektet):

- Lage entitetsklasse for dagbok-innlegg (engelsk: “DiaryEntry”)
- Dokumenter klassen grundig, inkludert
 - Rolle/ansvar
 - Hvilke informasjon klassen holder på og hvilke datatyper som er valgt for hver informasjon og hvorfor (begrunnelse)
 - En vurdering av hvilke informasjon skal kun settes ved opprettelse av instans, og hvilken informasjon må kunne endres etter at instansen er opprettet
 - Hvordan klassen responderer på ugyldige data - hvilken strategi følger klassen (kaster unntak? setter dummy-verdi?)
- Implementer nødvendige felt i klassen, med egnede datatyper. Se eget avsnitt under for tips til håndtering av dato og tid.
- Lag enhetstester for å teste at klassen for dagbokinnlegg fungerer som den skal og er robust. Husk både *positive*- og *negative* tester.
- Kjør CheckStyle på koden din med Google-reglene. (Se [wiki-siden for IDATx1003](#) for hvordan installere CheckStyle for IntelliJ)
- Opprett en klasse som på sikt skal bli brukergrensesnittet (UI) og dermed ta seg av all brukerinteraksjon. Opprett to metoder; `start()` og `init()`.
- Bruk `start`-metoden til å skrive enkel testkode for å teste at dagbokinnlegg-klassen fungerer i henhold til kravspesifikasjonen ved for eksempel å opprette 3-4 instanser av klassen og skrive ut objektene. Det kan lønne seg å lage en egen metode for å skrive ut detaljene til et innlegg.
- Opprett til slutt en klasse som skal være selve applikasjonen, og som inneholder ***public static void main(String[] args)***-metoden.
- Opprett `main()`-metoden. La denne metoden lage en instans av UI-klassen og kall deretter metodene `init()` og `start()` til UI-objektet.

Husk å utføre en commit (innsjekk) til lokalt Git-repository med jevne mellomrom. Husk å alltid legge til en kommentar til innsjekken. Husk også å utføre push til GitHub med jevne mellomrom for

å oppbevare koden trygt i skyen. Dette er svært viktig i tilfelle man får problemer med datamaskinen.

Håndtere dato og tid

Det finnes flere måter å håndtere dato og tid på i Java. For JDK v21 er den anbefalte måten for både å håndtere dato og tid i en og samme datatype, å bruke klassen [LocalDateTime](#) (klikk på lenken for å se JavaDoc for klassen).

Gjør dere godt kjent med denne klassen ved å lese dokumentasjonen, se på eksempelkode, lage enkel testkode, gjerne i CodePad'en i BlueJ, og spør også gjerne KI (men vær kritisk til svarene).

4.2 Rapport

Begynn på rapporten allerede nå:

- Sett dere inn i rapportmalen
- Fyll ut forside, og skriv innledningen
- Lag et UseCase-diagram som viser hvilken funksjonalitet løsningen skal tilby en bruker.
- Start på kapitlene om teori og metode.
- I resultat-kapitlet kan dere allerede nå gi en kort beskrivelse av hvilke klasser dere ser for dere å lage i prosjektet, og vise et klassediagram som viser avhengigheten mellom dem.

Del 2 (av 3)

Register over dagbokinnlegg

5 Mappe Del 2 av 3

I denne delen av mappen skal du implementere registeret som skal holde på alle innleggene i dagboken i (nivå 1) og register over forfattere (nivå 2) samt tilhørende enhetstestklasse(r) som tester registerklassen(e) både med negative og positive tester.

5.1 Nivå 1 - Minimal Viable Product (MVP)

Implementer en klasse som er ansvarlig for å holde på en samling av dagbokinnlegg med tilhørende funksjonalitet. Du skal selv:

- Velge navn på klassen som skal representere registeret.
- Vurdere hvilken klasse fra Java SDK du tenker er passende å bruke for å lagre alle innleggene i (ArrayList, HashSet, HashMap og så videre). Husk å begrunn i rapporten hvorfor du valgte nettopp denne klassen fra SDK'en.

Fra listen over funksjonelle krav fra kravspesifikasjonen i Del 1 (3.3.2), bør register-klassen som minimum ha støtte for følgende funksjonalitet:

- En metode for å legge til et dagbokinnlegg.
- En metode for å søke opp et innlegg på en gitt dato. Her må du også vurdere om du skal returnere det første innlegget, eller alle innlegg på en bestemt dato.
- En metode for å slette et innlegg
- En metode som gir tilgang til samlingen av innlegg, slik at klassen som representerer brukergrensesnittet har mulighet til for eksempel å skrive ut alle innleggene. Innleggene bør da være sortert enten i stigende eller synkende rekkefølge basert på dato/tid.

I tillegg til disse metodene bør du selv tenke ut og implementere øvrige metoder som kan være nyttig i et slikt register.

NB! Beskriv i rapporten løsningen du valgte for å returnere tilgang til en sortert liste av innlegg. Beskriv hvordan du kom frem til løsningen (søkte på nett, ChatGPT, GitHub CoPilot, spurte en venn og så videre), og forklar med egne ord koden du har implementert (dette for å vise at du forstår koden du har implementert).

Dersom kode er inspirert av en løsning fra andre (for eksempel nettside) er det viktig at kilde henvises til og valg begrunnes.

5.1.1 Enhetstest av registerklassen for dagbokinnlegg

Opprett enhets-tester for å teste registerklassen. Husk at det er smart å planlegge på forhånd hva du skal teste i registerklassen og hvordan, og spesielt hvordan du skal teste for å få utført negativ testing. Dette er det lurt å skrive i JavaDoc'en til testklassen FØR du begynner å kode testene.

Husk både positiv(e) og negativ(e) tester. Kontroller gjerne dekningsgrad (test-coverage) for å sjekke at mest mulig av koden i register-klassen dekkes av testene. Bare husk at 100% dekning ikke er ensbetydende med at testene dine er gode.

5.1.2 Oppdatere applikasjonsklassen

Oppdater init() og start() metodene i klassen som representerer applikasjonen (og på sikt vil representere det tekstbaserte brukergrensesnittet):

- **init():** Her legger du inn all kode som er nødvendig for å initialisere applikasjonen ved oppstart, som f.eks. å opprette instansen av register-klassen.
- **start():** Oppdater denne metoden til å ta i bruk den nye register klassen. Opprett 3-4 innlegg i kode som du legger til i registeret. Test deretter noe av funksjonaliteten til registeret. Du kan f.eks. allerede nå tenke på å opprette en metode som skriver ut oversikt over innleggene til konsollet basert på registrerte innlegg. Det er lurt å lage en egen metode for å skrive ut denne oversikten. Vent med å implementere full meny med input fra brukeren. Dette kommer i del 3.

5.2 Nivå 2: Utvidet funksjonalitet – Eget register for forfattere m.m.

Det vil være svært sannsynlig at flere av dagbokinnleggene er skrevet av samme forfatter. I stedet for å lagre navnet til forfatteren som en tekst i et felt i dagbokinnlegget, kan forfatter være representert med egen klasse (engelsk: *Author*). Når et innlegg opprettes med en forfatter som ikke har skrevet innlegg før, bør det være mulig å opprette en ny forfatter og lagre forfatter i eget register.

Se kravspesifikasjon i 3.3.1 for innspill til hvilken funksjonalitet et register over forfattere bør ha for å imøtekomme de funksjonelle kravene. Dokumenter dine valg i rapporten.

5.2.1 Enhetstest av de nye klassene

Opprett enhets-tester for å teste både klassen for *forfatter* og registerklassen. Husk at det er smart å planlegge på forhånd hva du skal teste i registerklassen og hvordan, og spesielt hvordan du skal teste for å få utført negativ testing. Dette er det lurt å skrive i JavaDoc'en til testklassen FØR du begynner å kode testene.

Husk både positiv(e) og negativ(e) tester. Kontroller gjerne dekningsgrad (test-coverage) for å sjekke at mest mulig av koden i register-klassen dekkes av testene. Bare husk at 100% dekning ikke er ensbetydende med at testene dine er gode.

5.2.2 Oppdatere applikasjonsklassen

Oppdater `init()` og `start()` metodene i klassen som representerer applikasjonen med funksjonalitet fra Nivå 2:

- **init():** Legg til kode for å opprette registeret for forfattere.
- **start():** Oppdater denne metoden til å ta i bruk den nye klassen for register av forfattere. Opprett 3-4 forfattere i kode som du legger til i registeret. Opprett deretter 3-4 dagbokinnlegg med forfattere fra forfatter-registeret. Oppdater utskrift av et innlegg nå når forfatter er en egen klasse. Vent med å implementere full meny med input fra brukeren. Dette kommer i del 3.

Del 3 (av 3)

Ferdigstille applikasjon med brukergrensesnitt

6 Mappe del 3 av 3

Del 3 av mappen bygger videre på Del 1 og 2.

Basert på tilbakemeldingen du har fått på del 2: gjennomfør nødvendig refaktorisering (eng:refactoring) av koden din fra del 2.

Denne delen gjelder resten av mappe-perioden frem til innlevering i uke 50. Dere får tilbud om tilbakemelding på Mappen Del 3 i uke 47, etter dette er dere på egen hånd.

I denne siste delen av Mappeoppgaven skal du ferdigstille applikasjonen din og rapporten. I tillegg skal du nå sette ditt eget preg på applikasjonen:

- **Brukergrensesnitt:** Du skal implementere et brukergrensesnitt. Du må gjerne tenke alternativt her. Er en meny beste løsning, eller er kommandoer fra kommandolinjen et bedre alternativ (tilsvarende «world-of-зуul»-prosjektet)? Grafisk brukergrensesnitt er pensum først i Programmering 2, så det gis ingen ekstrapoeng dersom du velger å implementere et grafisk brukergrensesnitt.
Husk at her er det viktigere at du vektlegger brukervennlighet og et robust grensesnitt, enn et fancy brukergrensesnitt. Anbefales at du holder deg til pensum i emnet.
- Med utgangspunkt i opprinnelig kravspesifikasjon, hvilke endringer/forbedringer ville du ha gjort for at applikasjonen skal bli enda mer nyttig og brukervennlig for brukeren? Her har du lov til å avvike noe fra opprinnelig kravspesifikasjon og tilføre dine tanker og idéer. Løsningen din må selvsagt fortsatt oppfylle de grunnleggende funksjonelle brukerkrav, men du står fritt til å endre på designet av klasser, valg av datatyper om du mener forslagene oppgitt i oppgaveteksten ikke er gunstige, og legge til ny funksjonalitet.
- Med utgangspunkt i designprinsippene **coupling** og **cohesion**, hvilke eventuelle endringer vil du gjøre i ditt design? Kan det være aktuelt å introdusere flere klasser for å oppfylle cohesion-prinsippet bedre, eventuelt innføre flere metoder (delegere)?
- Hvordan møter din løsning prinsippet om lagdelt arkitektur?
- Beskriv i rapporten hva du har foreslått av endringer og hvordan du har valgt å implementere disse, og hvorfor disse endringene gir et bedre design i tråd med designprinsippene vi har lært i emnet.

Det er ikke krav om å lage enhetstester for klasser som håndterer brukerinteraksjon.

6.1 Hvilke læringsmål vi ønsker å teste i del 3

I tillegg til at du med mappen skal kunne demonstrere samtlige læringsmål i emnebeskrivelsen, vil vi i del 3 ha spesielt søkelys på:

- evnen til å levere et selvstendig arbeid
- evnen til å re-designe (refaktorere) egen løsning uten å tilføre nye feil
- evnen til å skrive fail-safe/robust kode
- evnen til å skrive kode som følger en etablert kodelstil (Google), og som har god lesbarhet og er dokumentert i henhold til anbefalte industristandarder (JavaDoc)
- evnen til å implementere et robust design etter prinsipper som modularisering, coupling, cohesion, responsibility driven design
- brukervennlighet/god brukerinteraksjon

6.2 Krav til del 3

6.2.1 Krav til Programmet/koden

Følgende krav gjelder (fortsatt) til denne delen av oppgaven:

- Koden skal følge en bestemt kodelstil (fortrinnsvis Google sin stil)
- Kodelstilen skal verifiseres med CheckStyle-plugin (for IntelliJ, VSCode osv) og vise ingen/minimalt med regelbrudd ved levering.
- Klassene samt alle metoder og variabler (felt, parametere, lokale variabler) skal ha gode, beskrivende navn som tydelig gjenspeiler hvilken tjeneste en metode tilbyr, eller hvilken type verdi variablene representerer/holder på.
- Alle navn på klasser, metoder og variabler skal være på engelsk
- Koden skal være dokumentert (på engelsk) iht standarden for JavaDoc (se hvordan JDK'en er dokumentert, for inspirasjon og som referanse)

Du velger selv hvilken IDE (utviklingsverktøy) du vil bruke på prosjektet (IntelliJ, VS Code, Eclipse etc), men prosjektet skal være satt opp som et Maven-prosjekt.

6.2.2 Git og GitHub

Prosjektet ditt skal ha både et lokalt repository, og være koblet til et sentralt repository på GitHub.

Underveis i prosjektet skal du:

- Foreta hyppige commits til lokalt repository med gode commit-meldinger, samt utføre push til sentralt repository.
- Bruke tagger for å markere releaseversjoner

Prosjektet skal til slutt pushes i sin helhet til sentralt repository (GitHub), i tillegg til å levers som ZIP-fil i Inspira.

6.3 Krav til rapport

Rapporten fra del 2 videreføres med tilbakemeldingene du fikk fra LA/Faglærer.

Rapporten skal være maks 2500 ord!

Til samtalen for tilbakemelding på del 3:

- bør rapporten i størst mulig grad ferdigstilles. Kapitlene resultat og drøfting skal nærme seg ferdigstilling, tilstrekkelig til at det er mulig å gi tilbakemelding.
- det er viktig at dere dokumenterer opprinnelige funksjoner/klasser contra refaktorerte funksjoner/klasser, og ikke minst hvorfor du valgte å refaktorere.

6.3.1 Noen kommentarer til rapportmalen

Rapportmalen dere har fått utlevert, er egentlig en mal som også benyttes til Bachelor oppgaver 3. året. Målet med å gi denne allerede i 1. semester, er at dere skal bli vant til å tenke **akademisk rapport** fra starten av, slik at det å skrive en slik rapport forhåpentligvis blir enklere for dere når dere kommer til Bachelor-oppgaven.

Alle akademiske/vitenskapelige rapporter følger omtrent samme mønster, så også denne malen:

1. **Innledning:** Her skriver dere kort om prosjektet som rapporten omhandler.
2. **Bakgrunn – teoretisk grunnlag:** All utvikling er basert på teorier og bakgrunnsinformasjon (f.eks. fra andre publikasjoner eller kilder/bøker). Her skal dere kort beskrive de **teoriene** som dere har benyttet når dere har løst mappen, og henviser til **kildene** der dere har teoriene ifra (bøker, nettressurser osv). IKKE skriv om teorier her som der IKKE har benyttet, og ikke omtaler senere i rapporten!
3. **Metode-design:** Her beskriver dere kort hvilke **verktøy** dere har benyttet i prosjektet (IDE'er, osv), samt en kort beskrivelse av **arbeidsmetoden** dere har jobbet etter.
4. **Resultat:** Dette er det viktigste og største kapitlet i rapporten. Her skal dere beskrive den endelige løsningen dere har kommet frem til. Bruk UML diagrammer som use-case diagram(er), klassediagram(er), sekvensdiagram(er) osv. Beskriv kort hver klasse i forhold til ansvar/rolle (ikke gå i detalj på alle metoder og felt). Begrunn ditt designvalg med referanse til teoriene du har beskrevet i teoridelen. Ikke drøft løsningen din i dette kapitlet, det kommer i neste kapittel
5. **Drøfting:** Nå er det tid for å reflektere over resultatene du har oppnådd i forhold til kravene som var gitt i oppgaven, og i forhold til teoriene om god design og robust løsning. Klarte du å lage en applikasjon som er designet med «high cohesion» og «low coupling»? Holder kodebasen din høy kvalitet og er veldokumentert? Beskriv og «bevis» ved å henviser til konkrete sider av løsningen din.
6. **Konklusjon:** En kort oppsummering av prosjektet som helhet. Maks en halv side.

Generelt:

- Pass på **den røde tråden** gjennom kapitlene; har du beskrevet en teori i teoridelen, så sørg for å henviser til denne i resultat- og/eller diskusjons-delen av rapporten. Teorier som ikke henvises til, skal ikke stå i rapporten.
- Terminologier osv: Dersom du *ikke* har benyttet deg av forkortelser/termer i rapporten, dropp kapitlet

6.4 Bruk av AI-verktøy som GitHub CoPilot og ChatGPT

Dersom du har benyttet deg av AI-verktøy som støtte/hjelp til å løse mappeoppgaven, skal dette dokumenteres tydelig både i koden (ved kommentarer) og i rapporten.

I rapporten skal du beskrive:

- Hvilke AI-verktøy du har benyttet (ChatGPT, GitHub CoPilot osv).
- Hva du har brukt AI-verktøy til (generere kode, stille spørsmål, hjelp til rapport osv)
- Der du har brukt AI-verktøy for hjelp til kode: dokumenter i rapporten de delene av koden som du har fått hjelp til av AI-verktøy, og beskriv hvorfor du mener foreslått kode løser problemet ditt, og hva koden gjør (du må forstå og kunne beskrive hva koden gjør).

7 Viktige sjekkpunkter

Når dere løser oppgaven, bør dere passe på følgende:

Kompilering/bygging og prosjektstruktur:

NB! Maven er valgfritt i Trondheim

- Er prosjektet et Maven-prosjekt/prosjekt med en ryddig og riktig katalogstruktur?
- Kan prosjektet kompilere uten feil (Maven compile)?
- Kan samtlige enhets-tester i prosjektet kjøres uten at de feiler (Maven test)?
- NB! Prosjektet SKAL kompilere ved å bruke Maven!

Versjonskontroll med git

- Er prosjektet underlagt versjonskontroll med sentralt repo (på GitHub)?
- Sjekkes det inn (commit) jevnlig?
- Beskriver commit-meldingene endringene på en kort og konsis måte?
- Benyttes grener («branches») på en hensiktsmessig måte?
- Benyttes **tags** på riktig måte i forhold til versjonen i pom.xml og på applikasjonen?

Enhetstester

- Har enhetstestene beskrivende navn som dokumenterer hva testene gjør?
- Er testene godt nok dokumentert til at det klart går frem hva som testes, og hva forventet resultat av testene er?
- Følger de mønstret Arrange-Act-Assert (AAA)?
- Tas det hensyn til både positive og negative tilfeller?
- Er testdekningen fornuftig? (Enkelte klasser egner seg ikke for enhetstester, som f.eks. klassen som representerer UI).

Kodekvalitet

- Er koden godt dokumentert i henhold til JavaDoc-standard? (NB! samtlige metoder som er public **skal** dokumenteres, også enkle metoder som get()- og set()-metoder).
- Er koden robust (unntakshåndtering, validering med mer)?
- Har variabler, metoder og klasser beskrivende navn?
- Er klassene gruppert i en logisk pakkestruktur?
- Har koden god struktur, med **løse koblinger** og **høy kohesjon**?
- Benyttes funksjonell programmering (lambda og streams) på en hensiktsmessig måte der det er naturlig?
- Benyttes verktøy som for eksempel CheckStyle for sjekk av kodelstil og dokumentasjon?

7.1 Nyttige referanser

- Kodekvalitet og beste praksis:
<https://www.ntnu.no/wiki/display/idadx1001/Kodekvalitet+og+beste+praksis>
- Kodelstilsjekk med CheckStyle og SonarLint for IntelliJ:
<https://www.ntnu.no/wiki/display/idadx1001/CheckStyle+and+SonarLint+for+IntelliJ>

- Kildekodekontroll med GitHub/GitLab:
<https://www.ntnu.no/wiki/pages/viewpage.action?pageId=235998273>
- JUnit-testing og Maven:
<https://www.ntnu.no/wiki/pages/viewpage.action?pageId=240747880>

A Vedlegg

Her finner dere lenke til nyttige ressurser, samt tips og triks.

A.1 Nyttige verktøy

For en oversikt over nyttige verktøy til bruk ved utvikling av programvare, se vår Wiki:

<https://www.ntnu.no/wiki/display/idadx1001>. (Se under menyen «Verktøy og IDE-er»).

A.1.1 UML-modellering

PlantUML Finnes som plugin til IntelliJ. Gratis/Open Source. Støtter samtlige UML-diagramtyper med mer. Se: <https://www.ntnu.no/wiki/display/idadx1001/Plugins+for+IntelliJ+IDE>

Hvor hører PlantUML-filene hjemme i Maven-prosjekt strukturen? Forslag:

- **src/doc/plantuml/**: legg PlantUML-filene her (de som ender på **".puml"**).
- **docs/uml/**: legg de genererte PNG-diagramfilene her.

B KI-deklarasjon

KI-deklarasjonen som *skal* leveres inn utfylt, er vedlagt på neste side. Last ned Word-malen og fyll den ut, og last opp generert PDF.

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten blitt anvendt KI-baserte hjelpemidler?

☐ Nei

☐ Ja

Hvis *ja*: spesifiser type av verktøy og bruksområde under.

Tekst

☐ **Stavekontroll.** Er deler av teksten kontrollert av:
Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinkai.ai eller lignende verktøy?

☐ **Tekstgenerering.** Er deler av teksten generert av:
ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

☐ **Skriveassistanse.** Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av:
ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis *ja* til anvendelse av et tekstverktøy - spesifiser bruken her:

Kode og algoritmer

☐ **Programmeringsassistanse.** Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultaterslik som figurer, tabeller eller tallverdier blitt generert av: *GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard* eller lignende verktøy?

Hvis *ja* til anvendelse av et programmeringsverktøy - spesifiser bruken her:

Bilder og figurer

☐ **Bildegenerering.** Er ett eller flere av bildene/figurene i rapporten blitt generert av: *Midjourney, Jasper, WriteSonic, Stability AI, Dall-E* eller lignende verktøy?

Hvis *ja* til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy

☐ **Andre KI-verktøy.** har andre typer av verktøy blitt anvendt? Hvis *ja* spesifiser bruken her:

☐ Jeg er kjent med NTNUs regelverk: *Det er ikke tillatt å generere besvarelse ved hjelp av kunstig intelligens og levere den helt eller delvis som egen besvarelse.* Jeg har derfor redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet